# Mini Project: Who's That Pokémon?

**Assigned:** 2026-06-17 · **Due:** 2026-06-18

## Purpose

- To practice consuming an **external API** (the PokéAPI) from your own code.
- To build an **Express server with multiple routes** that holds game state and returns JSON.
- To get comfortable with **status codes** and testing routes by hand (Insomnia, curl, Node).
- To have fun.

## Task

Build an **Express server** for a guessing game, modeled on how `deckofcardsapi.com` works: a first call starts a game and hands back a `game_id`, and every later call passes that `game_id` back.

The server picks a **random Pokémon** (using the **PokéAPI**, https://pokeapi.co/) and the player tries to name it by requesting **hints** and making **guesses**. The player interacts with the game purely through **API calls** (Insomnia, curl, or a Node script); no web page is required for the minimum.

### Core loop

1. **Start a game:** `GET /new` returns a `game_id` and how many hints are available.
2. **Ask for a hint:** `GET /hint/:game_id/:n` returns the nth piece of information.
3. **Make a guess:** `GET /guess/:game_id/:guess` replies right or wrong.
4. If wrong, the player can keep guessing or ask for more hints, **in any order**, until they get it.

### Routes (minimum)

- `GET /new` : start a game. Returns `{ game_id, hints }`.
- `GET /hint/:game_id/:n` : return hint number `n` (1-based) for that game.
- `GET /guess/:game_id/:guess` : return whether the guessed name is correct.

## Hints

Hints are revealed by **number**, and each number always maps to the **same piece of info** (so `/hint/1` is always the same kind of hint). A suggested order, from vague to revealing:

1. The Pokémon's **first listed move**
2. Its **second listed move**
3. Its **type(s)**
4. Its **height and weight**
5. A link to its **cry** audio file

You choose the exact list and length; just keep it **ordered and consistent**.

## The `game_id` (important)

The `game_id` should **not show which Pokémon it is** at a glance, but your server still needs to recover the Pokémon from it on every call.

A simple approach: pick a random Pokémon id, then **base64-encode** it to form the `game_id`; on later calls, **base64-decode** the `game_id` back into that id. Your server stays **stateless** and remembers nothing between calls.

```
// starting a game: encode a random id
const id = 25;
const game_id = Buffer.from(String(id)).toString("base64");

// later, on /hint or /guess: decode it back
const decoded = Number(Buffer.from(game_id, "base64").toString());
```

Honest caveat: base64 is **obfuscation, not security**: a curious player could decode it. That is fine for a guessing game, where cheating only spoils your own fun. The first tiered goal below shows a sturdier, stateful alternative.

## Errors

Return a sensible **status code** and message for bad input, for example:

- `GET /hint/:game_id/99` when only 5 hints exist: `400` with an error message.
- A `game_id` that does not decode to a valid Pokémon id (e.g. not a number, or out of range): `400`.

## Inspiration

Play with `deckofcardsapi.com` in Insomnia or a browser. Notice how the first call gives you a `deck_id`, and every later call (draw, shuffle) passes it back. Your server works the same way, with a `game_id`.

## Example Exchanges

Request on top, JSON response below; your exact shapes can differ.

**Start a game**

```
GET /new
{ "game_id": "a1b2c3d4", "hints": 5 }
```

**Ask for the first hint**

```
GET /hint/a1b2c3d4/1
{ "hint": 1, "category": "move", "value": "thunder-shock" }
```

**A wrong guess, then a right one**

```
GET /guess/a1b2c3d4/charizard
{ "correct": false }
```

```
GET /guess/a1b2c3d4/pikachu
{ "correct": true, "pokemon": "pikachu" }
```

**Asking for a hint that does not exist**

```
GET /hint/a1b2c3d4/99
400  { "error": "Hint 99 does not exist (max 5)" }
```

## Using a Different API

The game does not have to be Pokémon themed. **PokéAPI is just a free, robust default** with lots of data per item, which makes writing good hints easy.

If you would rather theme it differently, swap in another **free external API** (no key, rich data) and keep the same shape: `/new` picks a random item, ordered `/hint` routes reveal its fields,

and `/guess` checks the answer.

Whatever you pick, the **Express server, routes, and game flow** are what matter, not the subject.

## Pair Programming

This is a **paired** project: work with a partner, together in real time.

- **Driver:** types the code. **Navigator:** reviews, thinks ahead, and spots issues.
- **Switch roles often** so you both drive and both navigate.
- You share **one repo and one codebase**; both partners should understand every part.
- Commit as a pair, and **credit both partners** in your README.

## Requirements and Allowances

- Your code must be tracked in a **GitHub repo from the start**, with **regular commits and pushes** throughout the project's life cycle.
- You may use **boilerplate tools** (e.g. `yarn init`, `express`), but you must **extend / modify** them in meaningful ways.
- You may use **AI tools** at any stage (planning, building, debugging), but you must **disclose** which AI, how, and where you used them.
- You may use **third-party packages** (e.g. a uuid generator, a fetch helper); note which ones you used and why.

## Deliverables

- A link to your **GitHub repository**.
- A **working Express server** that satisfies the Core loop (new game, hints, guesses) and can be driven from **Insomnia / curl / Node**.
- A short **README** describing the routes, how to run the server, and your **AI-use disclosure**.

## Submission

- Push your final work to your GitHub repo.

- Submit the repo link via the **repo submission form** provided in slack.

## Tiered Learning Goals

Finished the minimum? Keep extending the **server**. These stay API-first; the React client is intentionally last.

- **Server-side game store:** instead of encoding the answer in the `game_id`, make it a **random token** and keep a **lookup on the server** from `game_id` to that game's data. This is how `deckofcardsapi` works, and it unlocks tracking state (hints used, guesses, scoring). Note: data in memory is lost on restart.
- **Pick a generation:** let `GET /new?generation=1` choose a random Pokémon from one generation (gen 1 is ids 1-151, gen 2 is 152-251, and so on).- **Game status route:** `GET /game/:game_id` reports hints revealed, guesses made, and whether it is solved.
- **Give up:** `GET /giveup` reveals the answer and ends the game.
- **Forgiving guesses:** accept case-insensitive and trimmed names; optionally tolerate small misspellings.
- **Guess with POST:** change `/guess` from a `GET` (name in the URL) to a `POST` with the guess in a **JSON body** (parsed by `express.json()`), e.g. `POST /guess/:game_id` with `{ "guess": "pikachu" }`.
- **Scoring:** compute a score from hints used and guesses made, and return it when the player wins.
- **Choose your hint:** support categories like `GET /hint/type` or `GET /hint/cry` instead of a fixed number order.
- **Document it with Swagger / OpenAPI:** describe your routes so others can try them in the browser.
- **Deploy** your server so others can play it over the internet.
- **A React client:** a small web app that plays the game against your server.
- **Autocomplete:** an autocomplete name input on that client.
- **A running score:** show streak / score in the client UI.